Technische Hochschule Nürnberg

Maschinelles Lernen

Prof. Dr. Tobias Bocklet

Wo sind wir?

- Intro
- kNN
- Evaluation von Klassifikatoren
- Hierarchisches Clustering & kMeans
- MFCCs
- LinReg/Polynomregression, Merkmalnormierung
- LogReg
- Neural Nets
- Backpropagation/Trainingsverfahren
- Convolutional Neural Networks
- PreTraining
- Support Vektor Maschinen
- Statistische Klassifikation
- …

Organisatorisches

- **Keine Vorlesung Mittwoch!**
- **Keine Vorlesung**

- **Übungen am Mittwoch finden statt: Implementierung Lineare Regression**

Heute

- **Neuronale Netze**
- Motivation
- Perceptron
- Der KI-Winter
- Deep Nets
- Training

ohm Technische Hochschule Nürnberg

Neuronale Netze

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

Fathers of the Deep Learning Revolution Receive ACM A.M. Turing Award

Bengio, Hinton and LeCun Ushered in Major Breakthroughs in Artificial Intelligence



Yoshua Bengio is a Professor at the University of



Geoffrey Hinton is VP and Engineering Fellow



Yann LeCun is Silver Professor of the Courant

"Erschreckend, wenn man das sieht": KI-Pionier Geoffrey Hinton über KI-Modelle

Ohne den britisch-kanadischen Forscher hätten sich tiefe neuronale Netze kaum durchgesetzt. Doch nun hinterfragt der Ex-Google-Mitarbeiter Hinton seine Arbeit.

Lesezeit: **19 Min.** In Pocket speichern 433



Wie sich der KI-Bildgenerator Midjourney Geoffrey Hinton in Form eines Aquarellbildes vorstellt. (Bild: Erstellt von Midjourney durch MIT Technology Review)

Geoffrey Hinton wins Nobel Prize in Physics

Oct 16, 2024

A U of T University Professor Emeritus, Hinton shared the honour with Princeton University's John J. Hopfield for discoveries and inventions that enable machine learning with artificial neural networks



Jürgen Schmidhuber @SchmidhuberAI · 7 Dec 2024

The #NobelPrize in Physics 2024 for Hopfield & Hinton turns out to be a Nobel Prize for plagiarism. They republished methodologies developed in #Ukraine and #Japan by Ivakhnenko and Amari in the 1960s & 1970s, as well as other techniques, without citing the original inventors.

Show more



Elon Musk @elonmusk

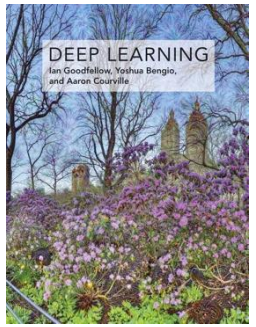
Schmidhuber invented everything

6:38 AM · Nov 23, 2023 · **92.7K** Views

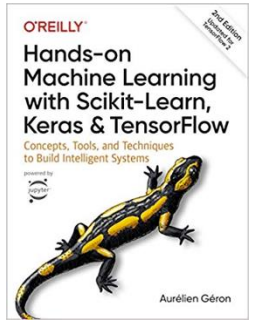
135 122 1.1K

Literatur

- Ian Goodfellow et. al: Deep Learning, MIT Press, 2016
  http://www.deeplearningbook.org/

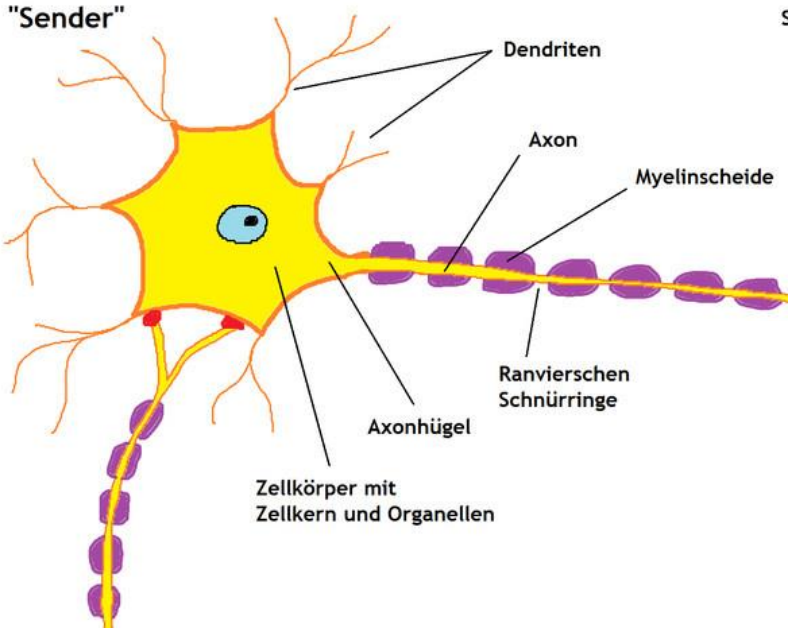


- Aurelien Geron: Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow, O'Reilly, 2019



Motivation – Biologische Motivation

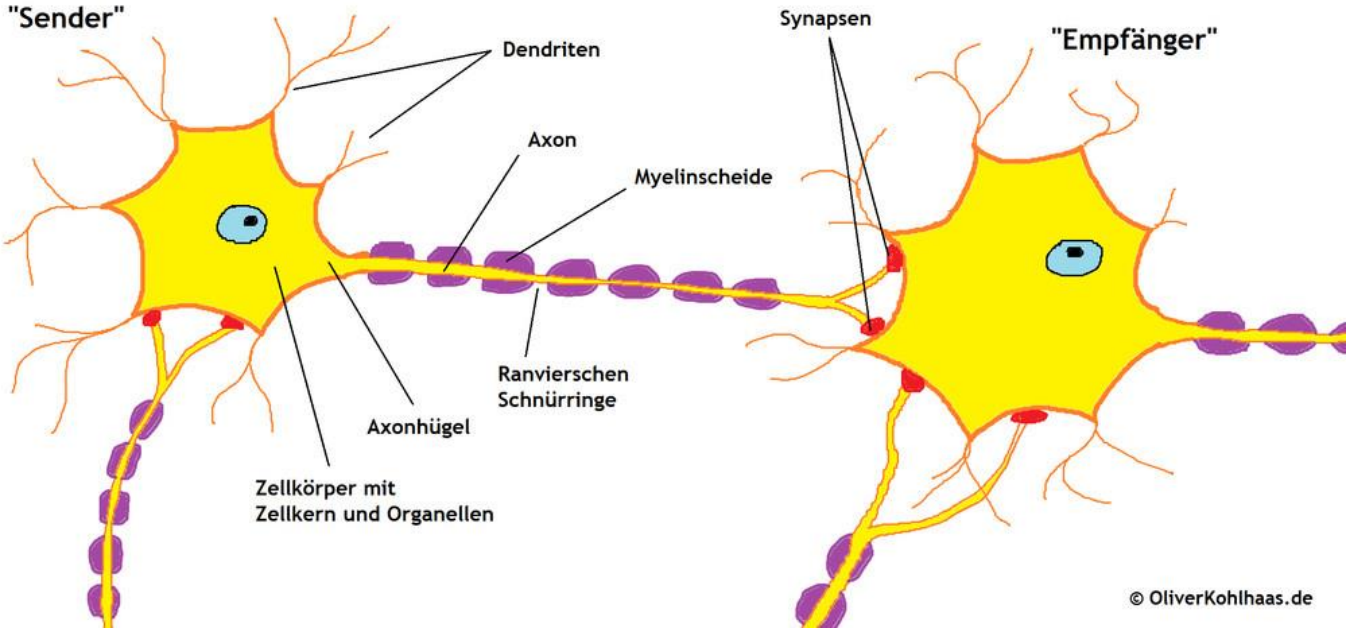
- 10^{11} Neurone im menschlichen Gehirn
- Neuron besteht aus
  - Dentriden
  - Synapsen
  - Zellkörper
  - Axon
- Schaltzeit: ca. 1ms
- 10^{14} Synapsen



https://www.oliverkohlhaas.de/neurobiologie-1/aufbau-nervenzelle/

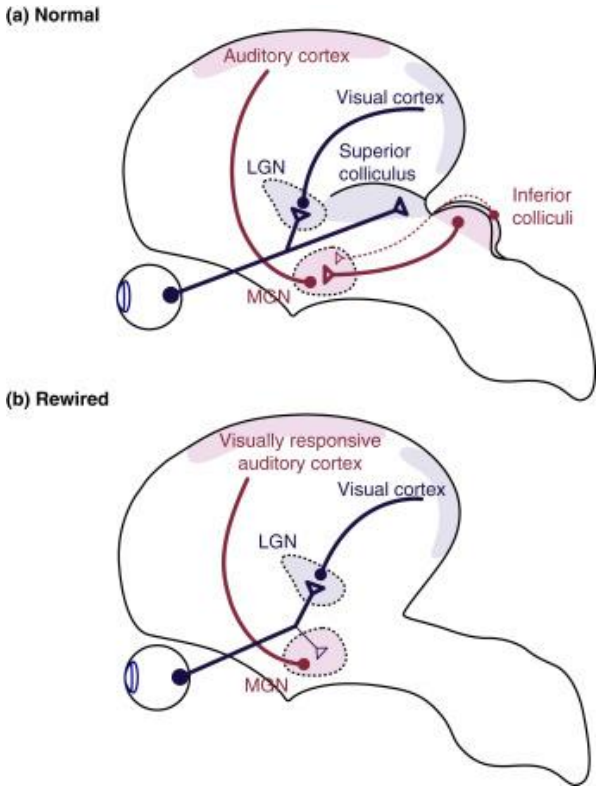
Motivation – Biologische Motivation

- 10^{11} Neurone im menschlichen Gehirn
- Neuron besteht aus
  - Dentriden
  - Synapsen
  - Zellkörper
  - Axon
- Schaltzeit: ca. 1ms
- 10^{14} Synapsen



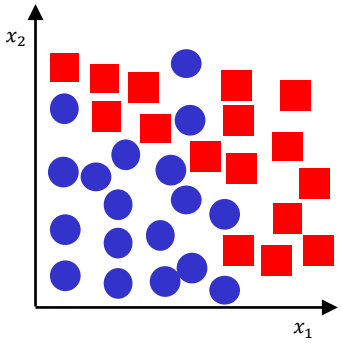
https://www.oliverkohlhaas.de/neurobiologie-1/aufbau-nervenzelle/

Theorie des „One learning“ Algorithmus



Rewiring the cortex: https://www.sciencedirect.com/topics/computer-science/receptive-field-property

Motivation: Nicht-Lineare Klassifikation



$$g(w_0 + w_1 x_1 + w_2 x_1^2 + w_3 x_1^2 x_2 + \\ + w_4 x_1^2 x_2^2 + w_5 x_1^2 x_2^3 + w_6 x_1^3 x_2 + \cdots)$$

Hochdimensional:

x_1= Größe

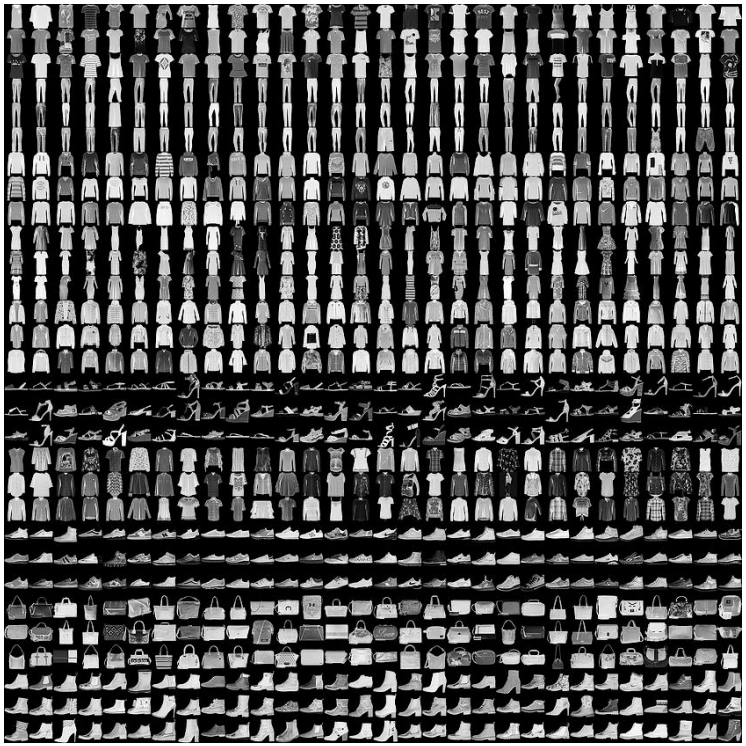
x_2= # Zimmer

x_3= # Stockwerke

…

x_{100}= Baujahr

What do you see?



Fashion-MNIST

D=28x28 = 768



(a) ImageNet Synset: One sample image from each category



(b) Corel-1000 Dataset: Sample images from each category



(c) Caltech-256 Dataset: One sample image from each category



(d) Caltech-101 Dataset: One sample image from each category

SW:300x200

d = 60000

RBG:300x200x3

d = 180000



Chihuahua or Muffin?

What do you see – Beispiel in der Übung

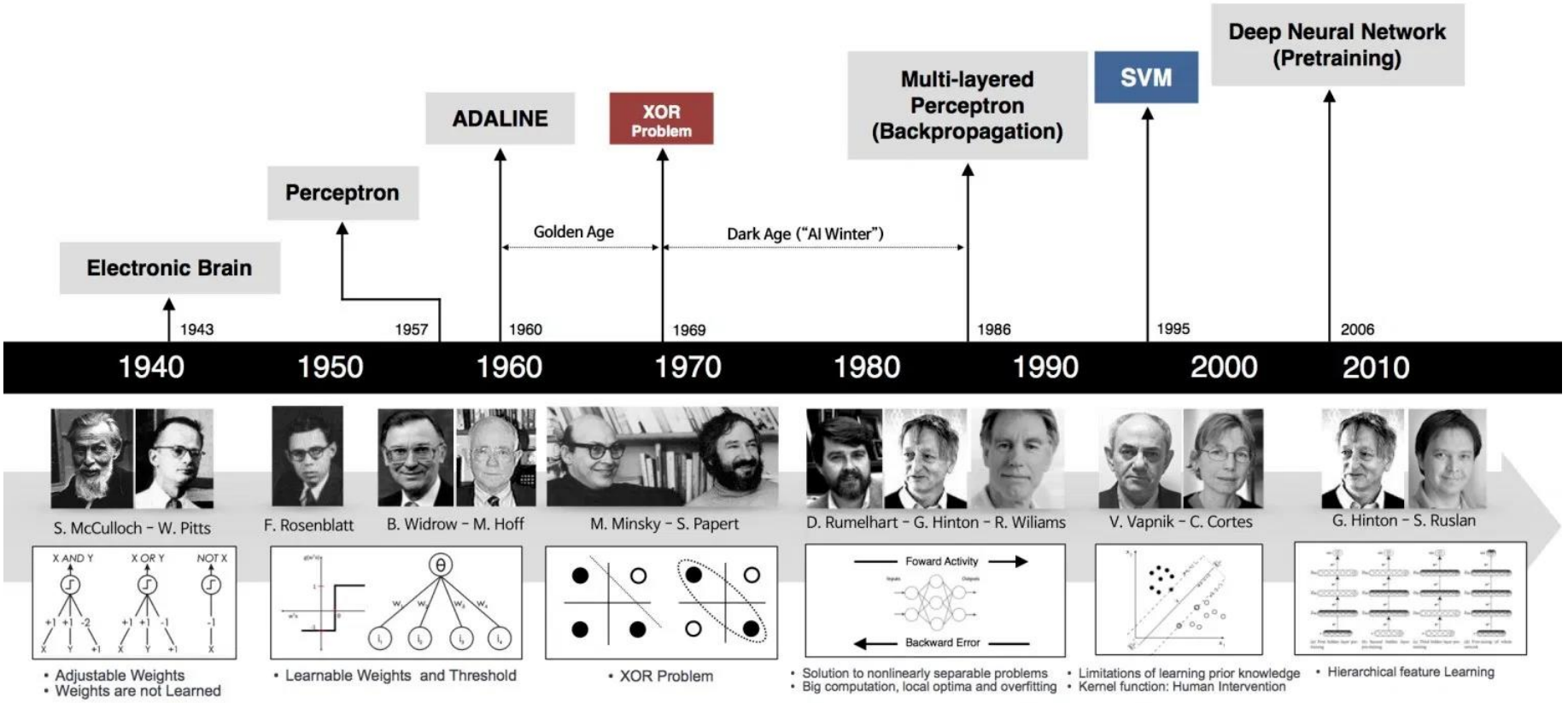
- Katze oder Hund?
- 25k Bilder
- **Wir bauen unser erstes Neuronales Netz**
- Brute force
  - Bildwerte als Eingabe
  - Ausgabe: Katze oder Hund



Neuronale Netze

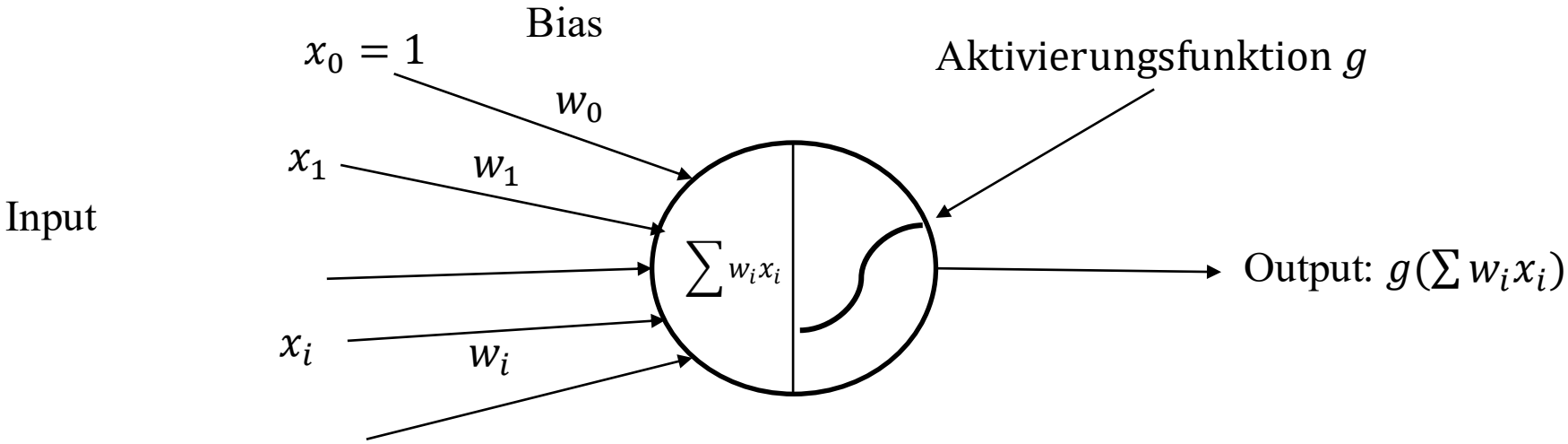
- Algorithmen die versuchen Handlungsweise des Gehirns nachzuahmen
- Erste Ansätze 1943
- Sehr populär in den 80er und 90er Jahren
- Ende der 90er keiner verwendete mehr Neuronale Netze
  → SVMs, Statistische Klassifikation, Hand-crafted Merkmale

Neuronale Netze



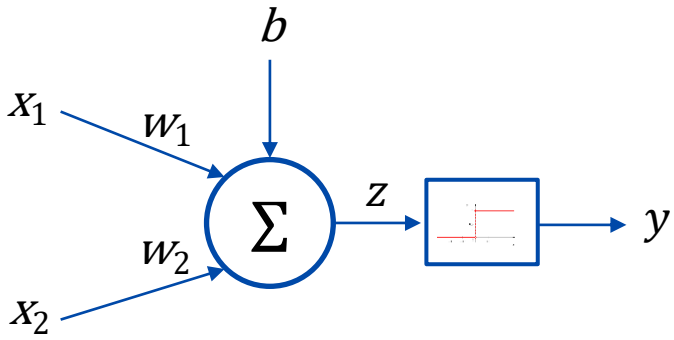
Perzeptron

- (Vereinfachte) Mathematische Interpretation der Neurone



Rosenblatt Perzeptron

- Andere Schreibweise mit Stufenfunktion zur Aktivierung:



$$z = \boldsymbol{w}^T \boldsymbol{x} + b$$

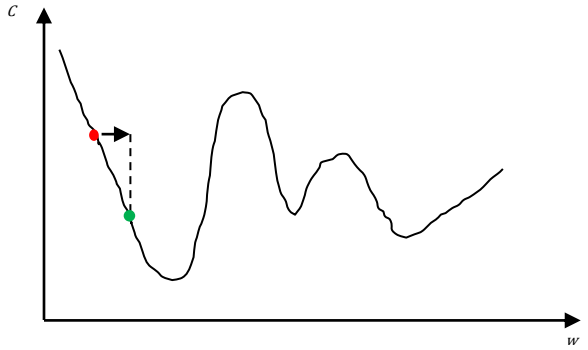
$$y = \begin{cases} 0, & z \leq 0 \\ 1, & z > 0 \end{cases}$$

Wie trainieren?

- Aufstellen einer Kostenfunktion C
- Durch Änderung der Gewichte sollen die Kosten C minimiert werden
- Folge dem Gradienten um Minimum in C zu finden

Gradientenabstiegsverfahren

- Aufstellen einer Kostenfunktion
- Durch Änderung der Gewichte sollen die Kosten C minimiert werden
- Folge dem Gradienten um Minimum in C zu finden



- Änderung der Gewichte $w_i \rightarrow w_{i_{neu}} = w_i + \Delta w_i$
- mit $\Delta w_i = -\varepsilon \frac{\partial C}{\partial w_i}$

- Herleitung der Delta-Regel

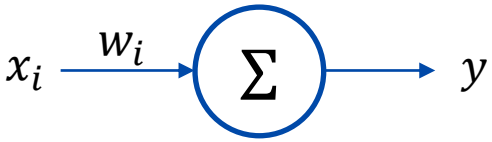
Delta-Regel – LinReg

Durch Einsetzen einer quadratischen Kostenfunktion:

$$C = \frac{1}{2}(\hat{y} - y)^2$$

Kettenregel:

$$\frac{\partial C}{\partial w_i} = \frac{dC}{dy}\ \frac{\partial y}{\partial w_i} = (-1)(\hat{y} - y)\, x_i$$



$$y = w_i x_i \qquad \frac{\partial y}{\partial w_i} = x_i$$

Delta-Regel:

$$\Delta w_i = -\varepsilon \frac{\partial C}{\partial w_i} = -\varepsilon x_i (\hat{y} - y)$$

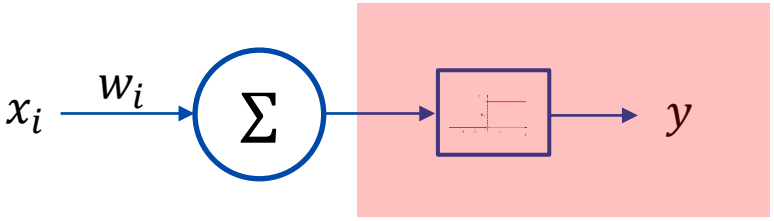
Delta-Regel – Perceptron

Durch Einsetzen einer quadratischen Kostenfunktion:

$$C = \frac{1}{2}(\hat{y} - y)^2$$

Kettenregel:

$$\frac{\partial C}{\partial w_i} = \frac{dC}{dy}\ \frac{\partial y}{\partial w_i} = (-1)(\hat{y} - y)\, x_i$$



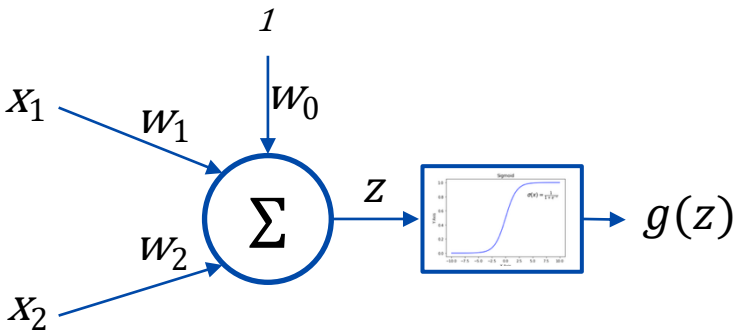
$$y = step(w_i x_i) \qquad \frac{\partial y}{\partial w_i} = x_i$$

Delta-Regel:

$$\Delta w_i = -\varepsilon \frac{\partial C}{\partial w_i} = -\varepsilon x_i (\hat{y} - y)$$

Minsky and Papert Perzeptron (1969)

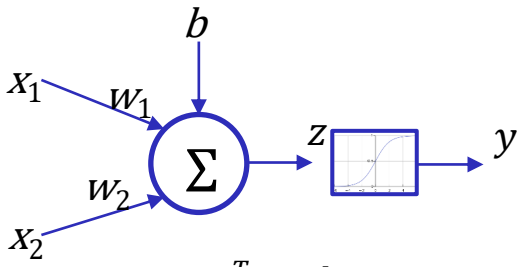
- Mit Sigmoidfunktion zur Aktivierung:



$$z = \boldsymbol{w}^T \boldsymbol{x} + b$$

$$y = g(z) = \frac{1}{1+e^{-z}}$$

Erweiterung mit Sigmoidaktivierungsfunktion



$$z = \boldsymbol{w}^T \boldsymbol{x} + b$$

$$y = g(z) = \frac{1}{1+e^{-z}} \qquad \frac{\partial y}{\partial z} = y(1-y)$$

$$\frac{\partial z}{\partial w_i} = x_i \quad \frac{\partial z}{\partial x_i} = w_i$$

$$\frac{\partial C}{\partial w_i} = \frac{\partial C}{\partial y}\frac{\partial y}{\partial w_i} = \frac{\partial C}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial w_i}$$

Binäre Kreuzentropie als Kostenfunktion

- $C = \frac{1}{2M} \sum_{i=1}^{M} Cost(h(x^{(i)}) - y^{(i)})$

- $$Cost(h(x), y) = \begin{cases} -\log(h(x)), & wenn\ y = 1 \\ -\log\big(1 - h(x)\big), & wenn\ y = 0 \end{cases}$$

- *y immer 0 oder 1*

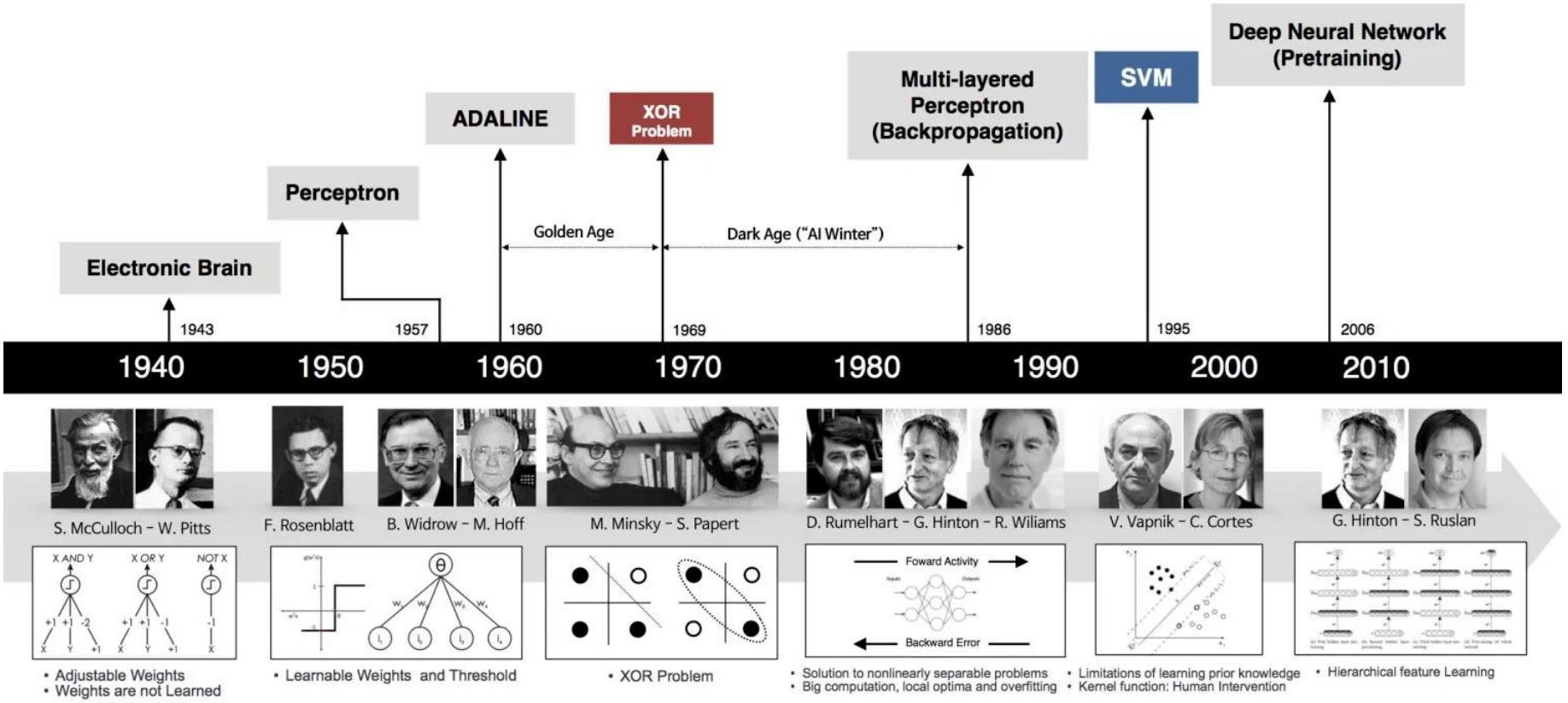
Kostenfunktion – Identisch zu Logistischer Regression

- $Cost(h(x), y) = -y \log\big(h(x)\big) - (1 - y) \log\big(1 - h(x)\big)$

- $C = \frac{1}{m} \sum_{i=1}^{m} Cost\big(h\big(x^{(i)}\big), y^{(i)}\big)$

$$= -\frac{1}{m} \sum_{i=1}^{m} y^{i} \log\big(h\big(x^{(i)}\big)\big) + \big(1 - y^{(i)}\big) \log\big(1 - h\big(x^{(i)}\big)\big)$$

Neuronale Netze



Das XOR-Problem: The dark age of AI

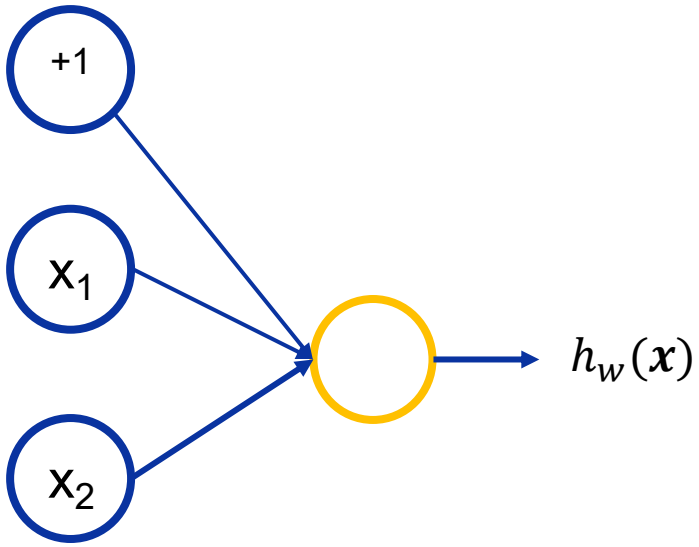
Logische Operationen mittels Perzeptron

- AND
- OR
- NOT

AND

$x_1, x_2 \in \{0,1\}$

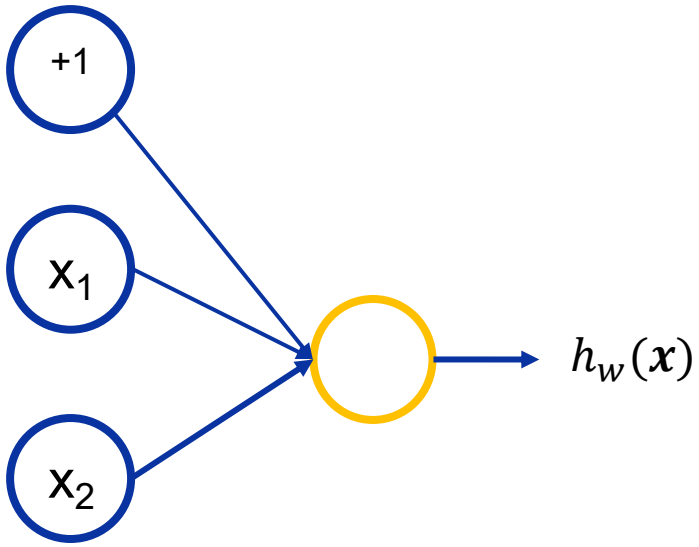
$y = x_1 \; AND \; x_2$



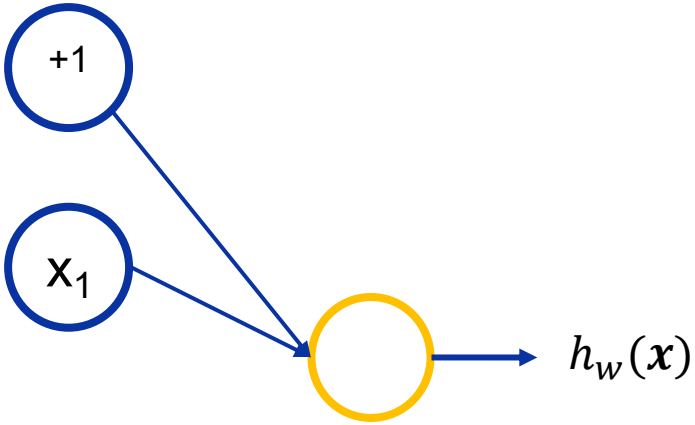
OR

$x_1, x_2 \in \{0,1\}$

$y = x_1 \, OR \, x_2$

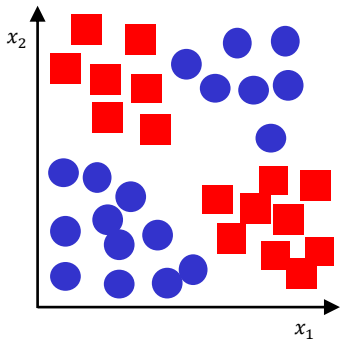


NOT

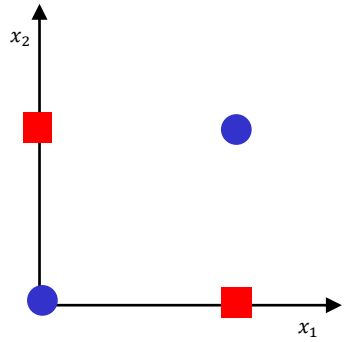


$$y = NOT\ x_1$$

Problem: Nicht-lineare Klassifikation: XOR/XNOR



x_1, x_2 sind binär (0 oder 1)



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

XOR

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

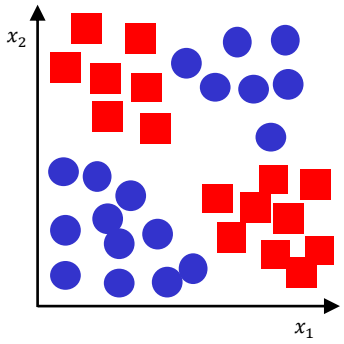
XNOR

$y = x_1 \text{ XOR } x_2$

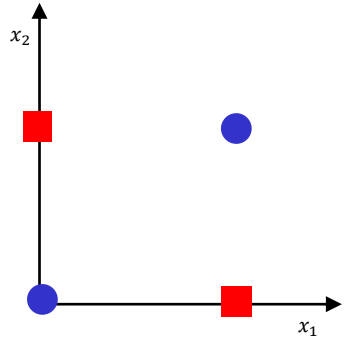
$y = x_1 \text{ XNOR } x_2$

Going Deep

Nicht-lineare Klassifikation: XOR/XNOR



x_1, x_2 sind binär (0 oder 1)



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

XOR

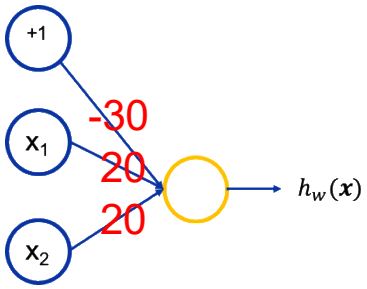
A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

XNOR

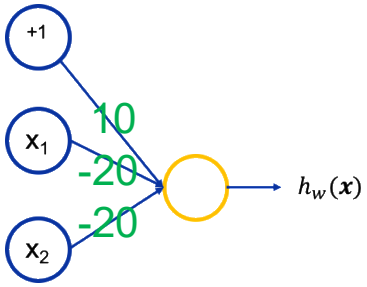
$y = x_1 \text{ XOR } x_2$

$y = x_1 \text{ XNOR } x_2$

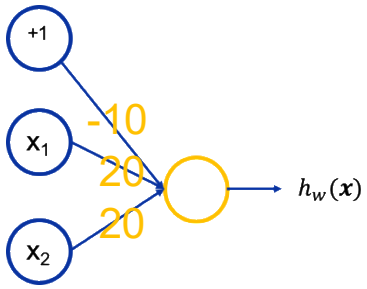
XNOR



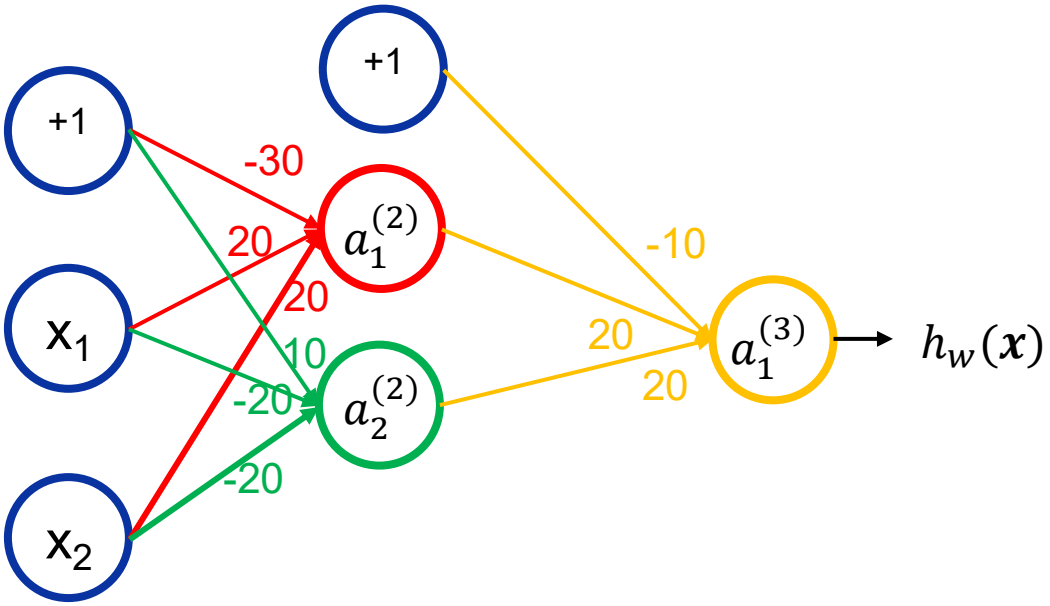
$y = x_1 \; AND \; x_2$



$y = NOT \; x_1 \; AND \; NOT \; x_2$



$y = x_1 \; OR \; x_2$

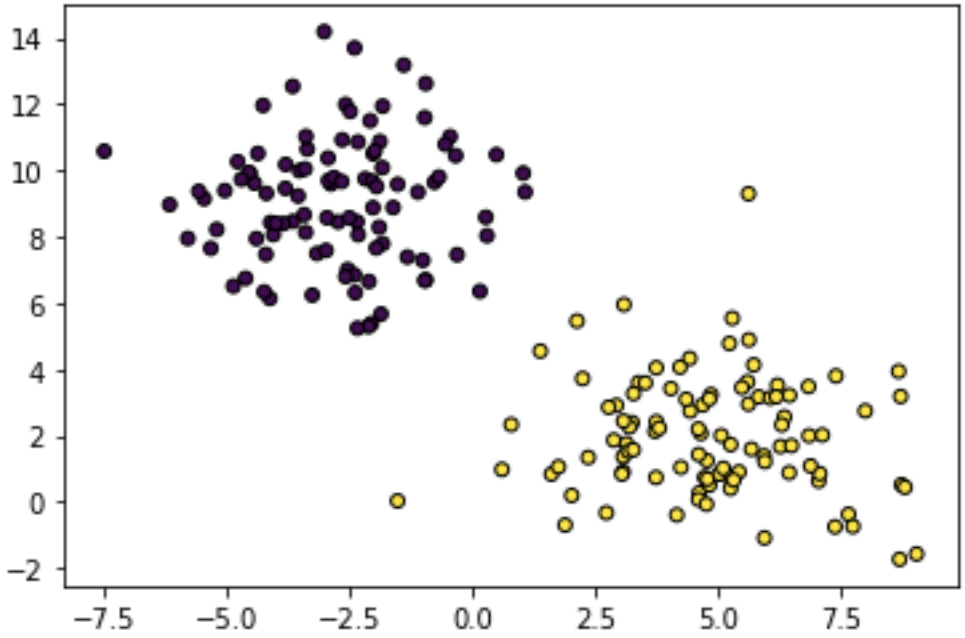


x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_w(x)$
0	0			
0	1			
1	0			
1	1			

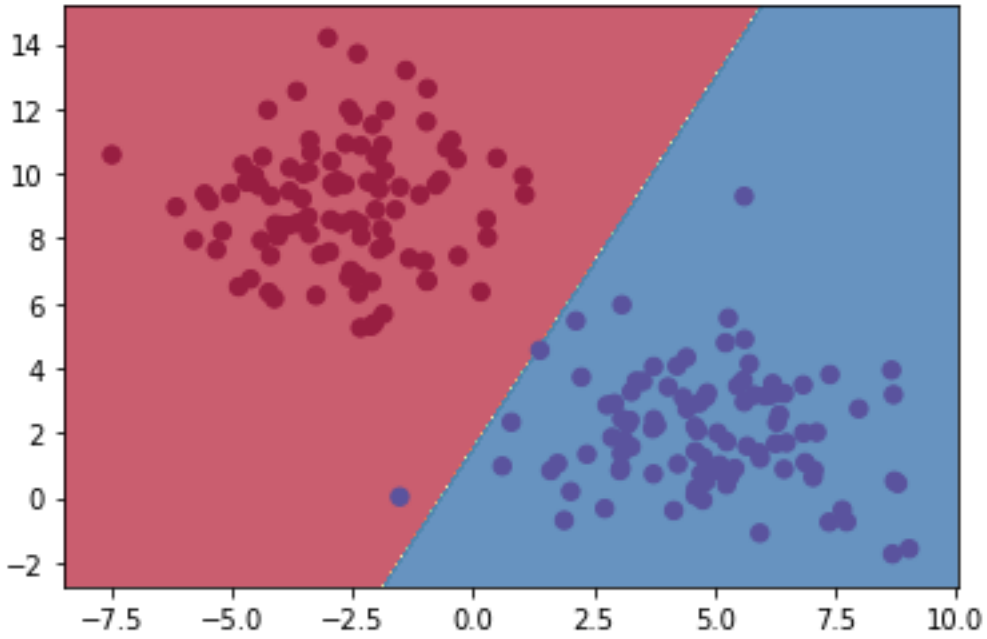
Ausblick

- Neuronale Netze
- Motivation
- Geschichte
- Vom Perzeptron zum Neuronalen Netz
- Deep Nets
- Trainieren Neuronaler Netze: Backpropagation
- Mehrklassen-Probleme
- Softmax
- Weitere Netztopologien

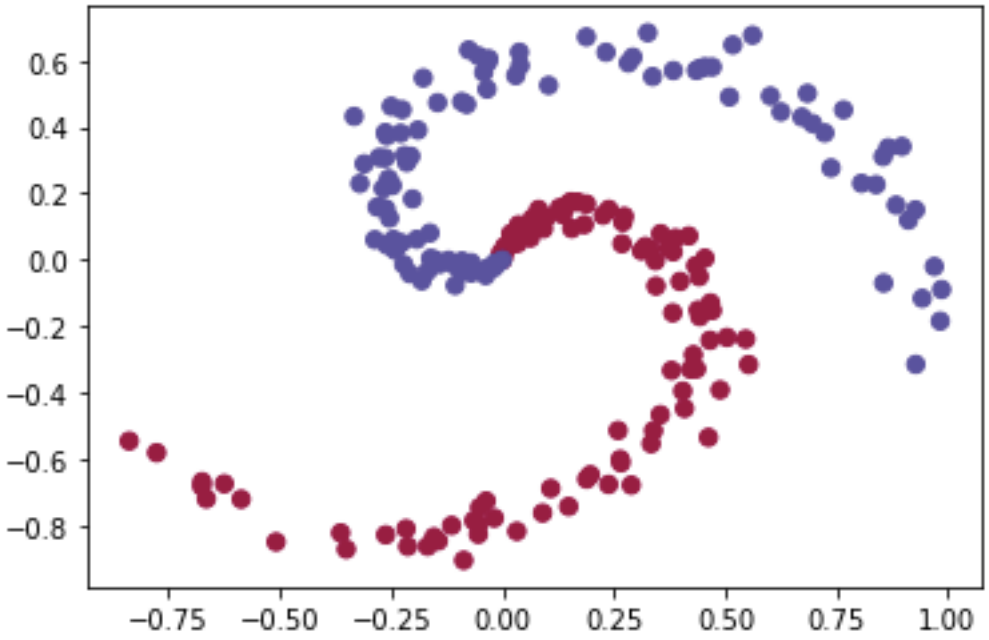
Wie sieht Klassifikation mit LogReg/Perceptron bei diesen Daten aus?



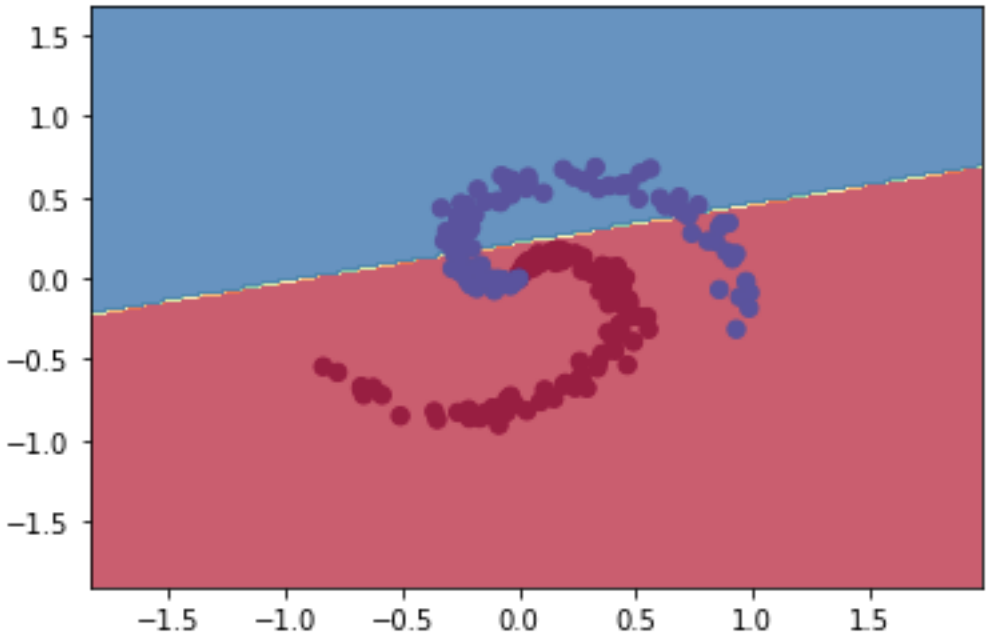
So:



Wie sieht Klassifikation mit LogReg/Perceptron bei diesen Daten aus?



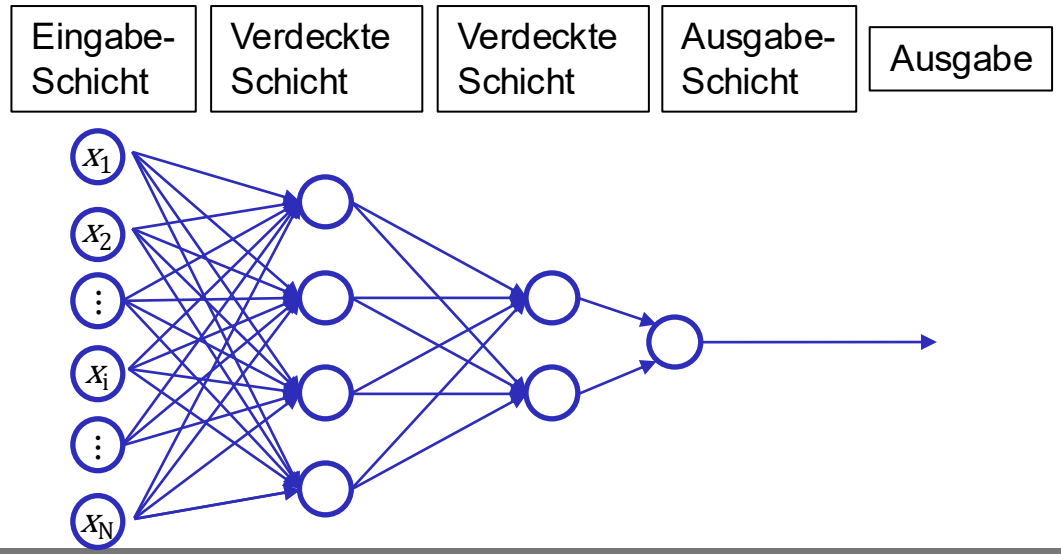
So:



Problem: Nur Lineare Trennbarkeit mit LogReg/Perceptron modellierbar

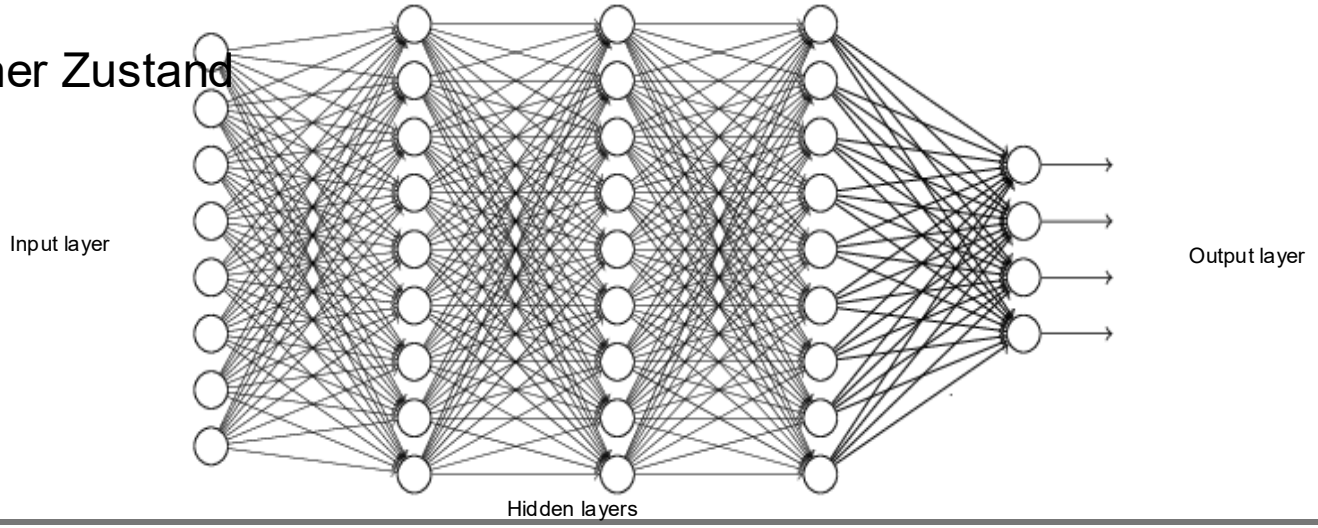
Going Deep – Neuronale Netze

- Beispieltopologie eines mehrschichtigen Neuronalen Netzes
- N Eingaben, eine Schicht mit 4 Knoten, weitere Schicht mit 2 Knoten, Ausgabeschicht
- Jeder (innere) Knoten repräsentiert ein Perceptron



Multi-Layer Perceptron (MLP)

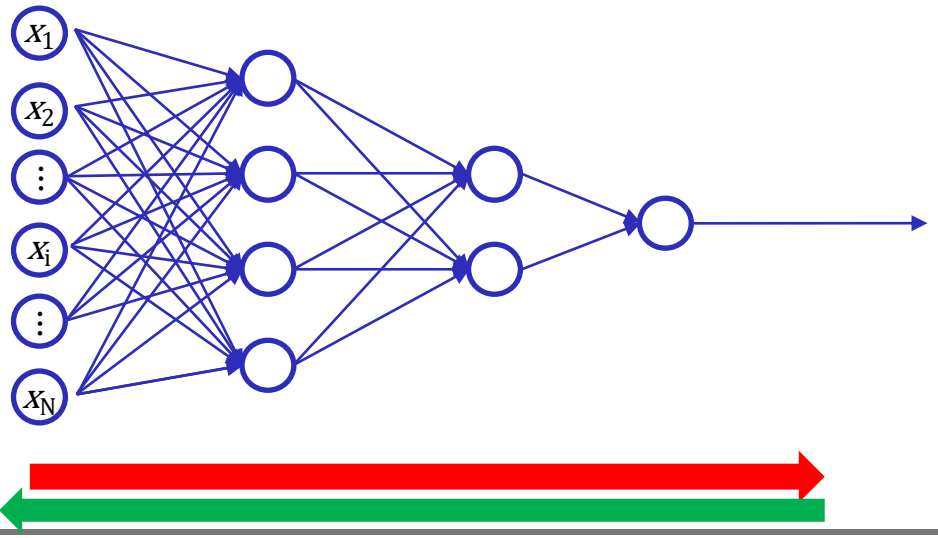
- Neurone in Schichten angeordnet
- Schicht n ist komplett mit layer $n+1$ verbunden: Fully Connected
- Keine Verbindungen innerhhalb einer Schicht
- Keine Verbindung zu anderen Schichten (keine Jumps, keine Feedbacks)
- Informationsfluss feed-forward
- Kein interner Zustand



Lernen Neuronaler Netze

- Vorwärtspfad: Lernstichprobe wird durch das Netz propagiert

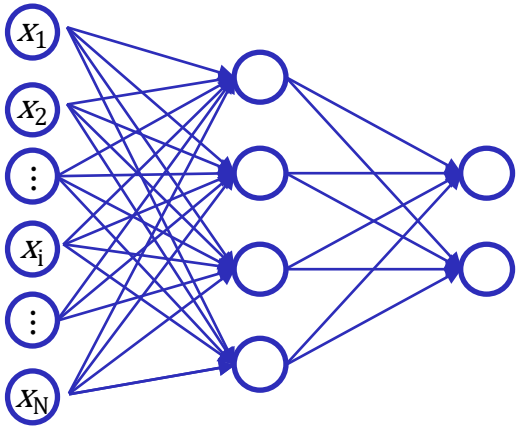
  → erzeugt Ausgaben y für jeden Merkmalevktor $\mathbf{x}$
- Fehler der Kostenfunktion wird berechnet
- Backpropagation: Bestimmen der partiellen Ableitungen für w



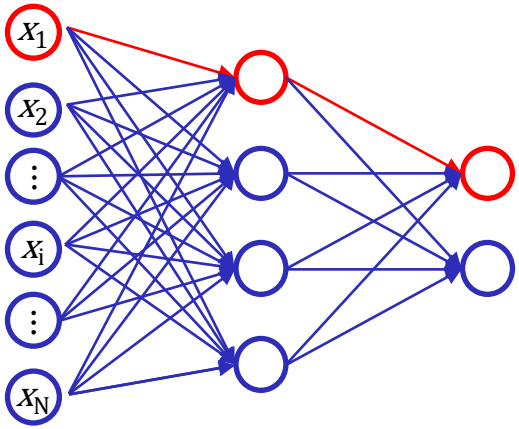
$$C = \frac{1}{2}(\hat{y} - y)^2$$

Lernen Neuronaler Netze

- Beispieltopologie eines mehrschichtigen Neuronalen Netzes
- N Eingaben, eine Schicht mit 4 Knoten, weitere Schicht mit 2 Knoten,
- Jeder Knoten repräsentiert ein Perceptron

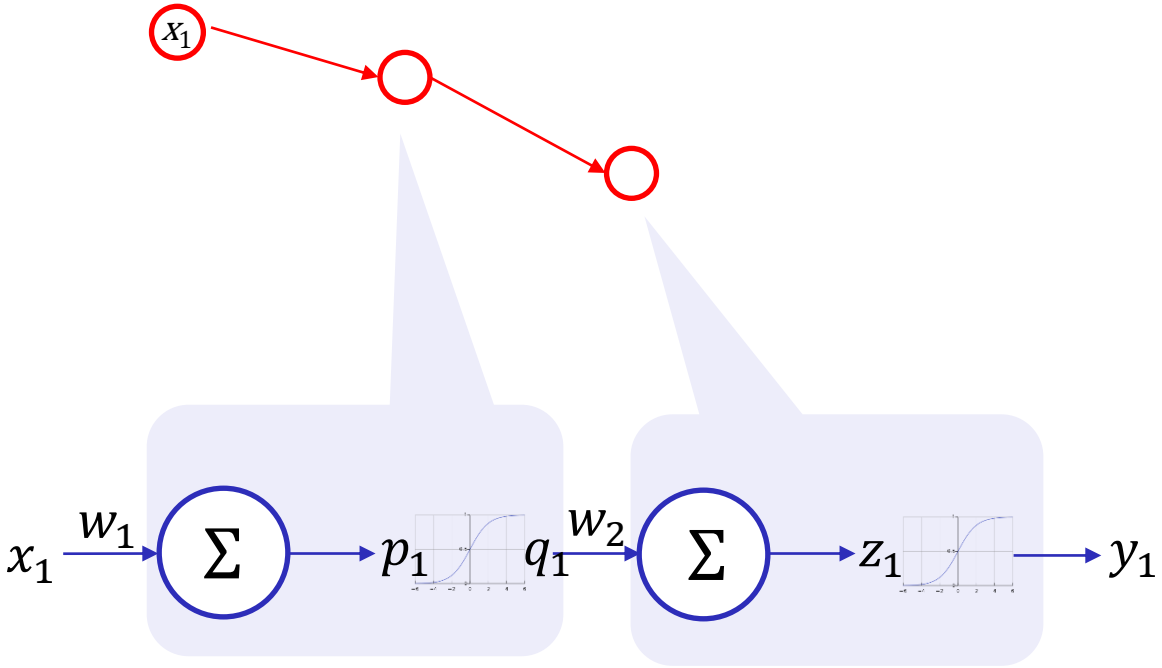


Backpropagation

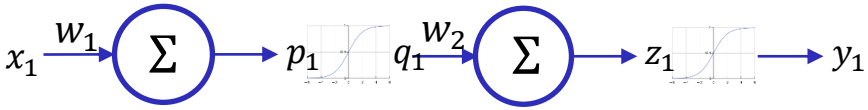


Rumelhart, D., Hinton, G. & Williams, R. Learning representations by back-propagating errors. *Nature* **323,** 533–536 (1986). https://doi.org/10.1038/323533a0

Backpropagation – Mehrschichtig



Backpropagation – Mehrschichtig



Spickzettel:

$$C = \frac{1}{2}(\widehat{y_1} - y_1)^2$$

$$y_1 = \varphi(z_1) = \frac{1}{1 + e^{-z_1}}$$

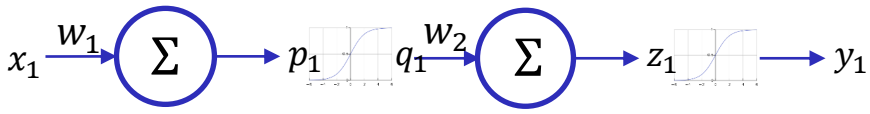
$$\frac{\partial C}{\partial y_1} = -(\widehat{y_1} - y_1)$$

$$\frac{\partial y_1}{\partial z_1} = y_1(1 - y_1)$$

$$\frac{\partial z_1}{\partial w_2} = q_1$$

$$\frac{\partial z_1}{\partial q_1} = w_2$$

Backpropagation – Mehrschichtig



$$\frac{\partial C}{\partial w_2} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial w_2} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial z_1}\frac{\partial z_1}{\partial w_2} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial z_1} q_1$$

$-(\widehat{y_1} - y_1)$, $y_1(1-y_1)$, q_1

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial z_1}\frac{\partial z_1}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_1} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial z_1}\frac{\partial q_1}{\partial p_1} w_2 x_1$$

w_2, $q_1(1-q_1)$, x_1

Spickzettel:

$$C = \frac{1}{2}(\widehat{y_1} - y_1)^2$$

$$y_1 = \varphi(z_1) = \frac{1}{1+e^{-z_1}}$$

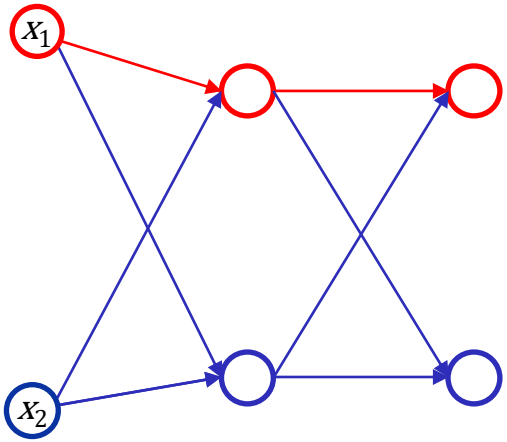
$$\frac{\partial C}{\partial y_1} = -(\widehat{y_1} - y_1)$$

$$\frac{\partial y_1}{\partial z_1} = y_1(1-y_1)$$

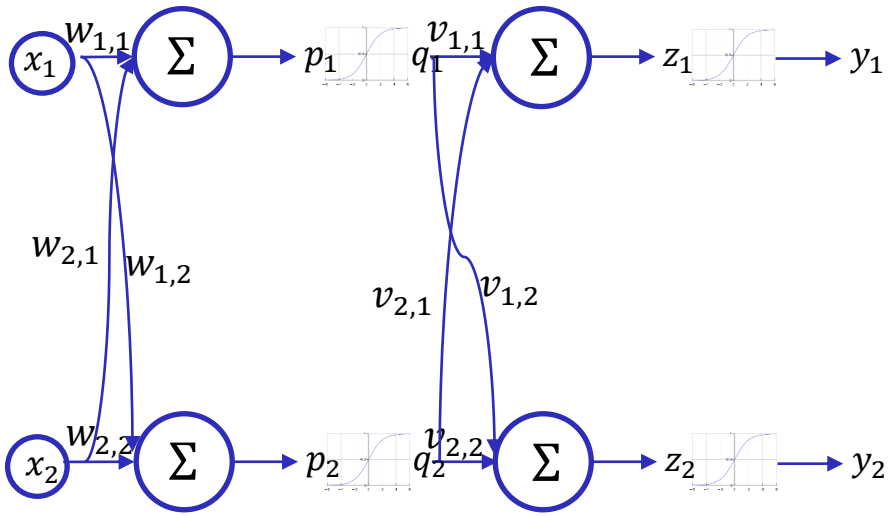
$$\frac{\partial z_1}{\partial w_2} = q_1$$

$$\frac{\partial z_1}{\partial q_1} = w_2$$

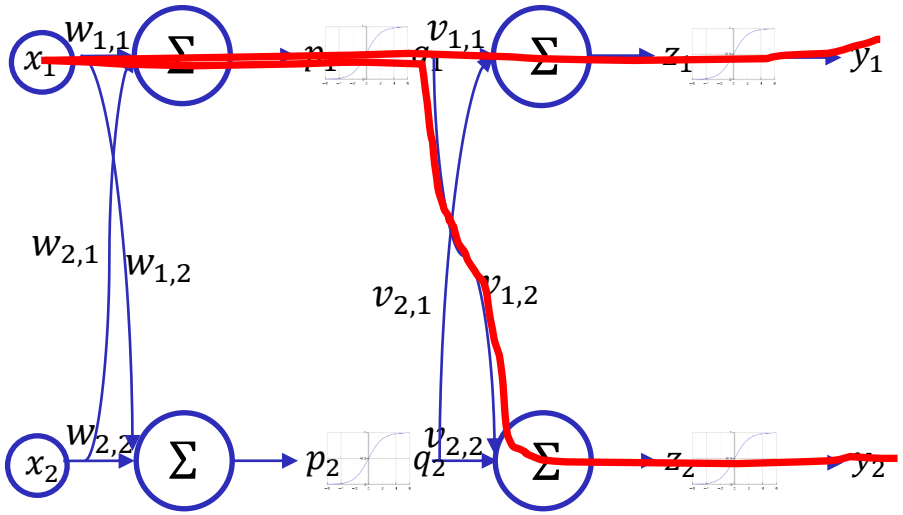
Backpropagation – Multi-dimensionale Erweiterung



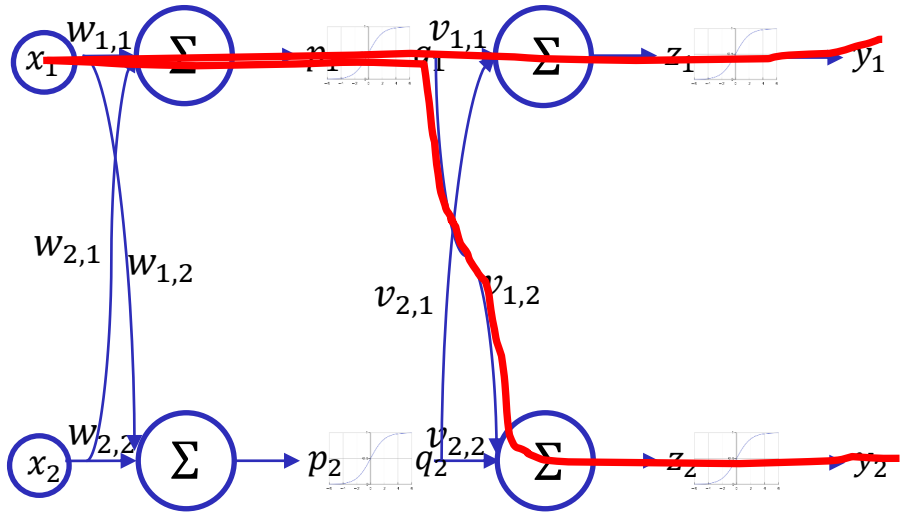
Backpropagation – Multi-dimensionale Erweiterung



Backpropagation – Multi-dimensionale Erweiterung



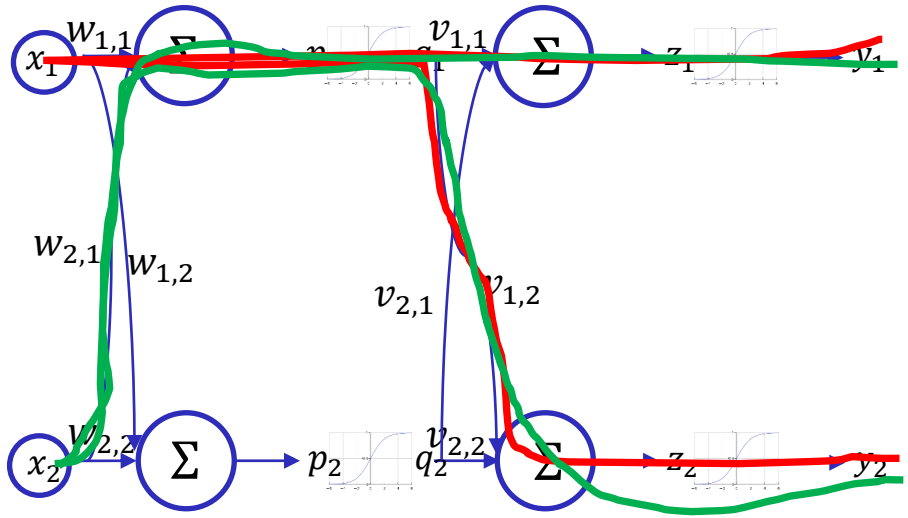
Backpropagation – Multi-dimensionale Erweiterung



$$\frac{\partial C}{\partial w_{1,1}} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial z_1}\frac{\partial z_1}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{1,1}}$$

$$+\frac{\partial C}{\partial y_2}\frac{\partial y_2}{\partial z_2}\frac{\partial z_2}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{1,1}}$$

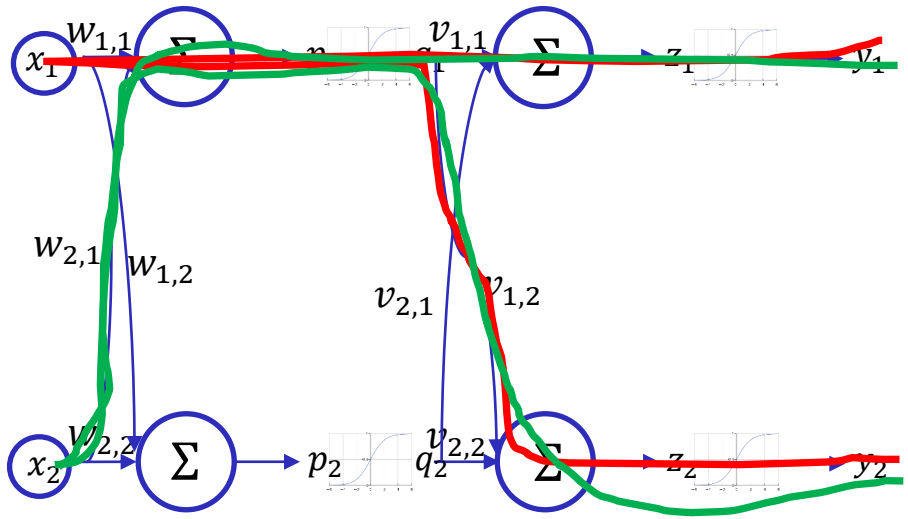
Backpropagation – Multi-dimensionale Erweiterung



$$\frac{\partial C}{\partial w_{1,1}} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial z_1}\frac{\partial z_1}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{1,1}} + \frac{\partial C}{\partial y_2}\frac{\partial y_2}{\partial z_2}\frac{\partial z_2}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{1,1}}$$

$$\frac{\partial C}{\partial w_{2,1}} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial z_1}\frac{\partial z_1}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{2,1}} + \frac{\partial C}{\partial y_2}\frac{\partial y_2}{\partial z_2}\frac{\partial z_2}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{2,1}}$$

Backpropagation – Multi-dimensionale Erweiterung



$$\frac{\partial C}{\partial w_{1,1}} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial z_1}\frac{\partial z_1}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{1,1}} + \frac{\partial C}{\partial y_2}\frac{\partial y_2}{\partial z_2}\frac{\partial z_2}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{1,1}}$$

$$\frac{\partial C}{\partial w_{2,1}} = \frac{\partial C}{\partial y_1}\frac{\partial y_1}{\partial z_1}\frac{\partial z_1}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{2,1}} + \frac{\partial C}{\partial y_2}\frac{\partial y_2}{\partial z_2}\frac{\partial z_2}{\partial q_1}\frac{\partial q_1}{\partial p_1}\frac{\partial p_1}{\partial w_{2,1}}$$

Anmerkungen

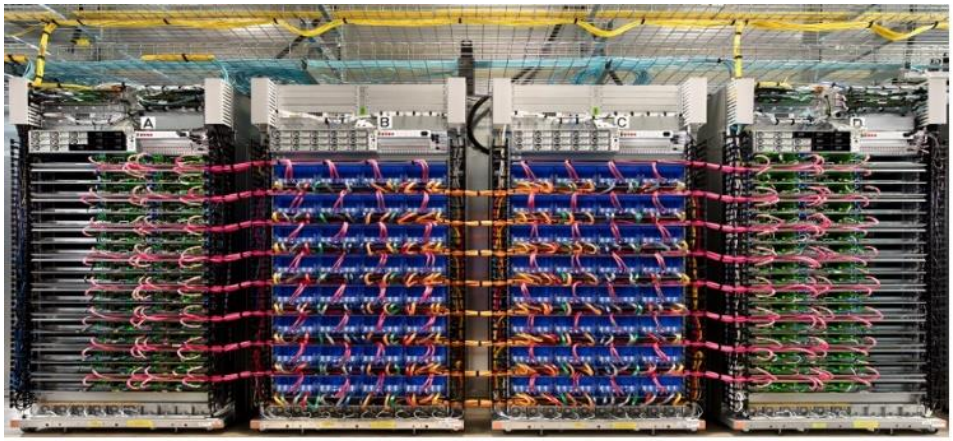
Vorteile:

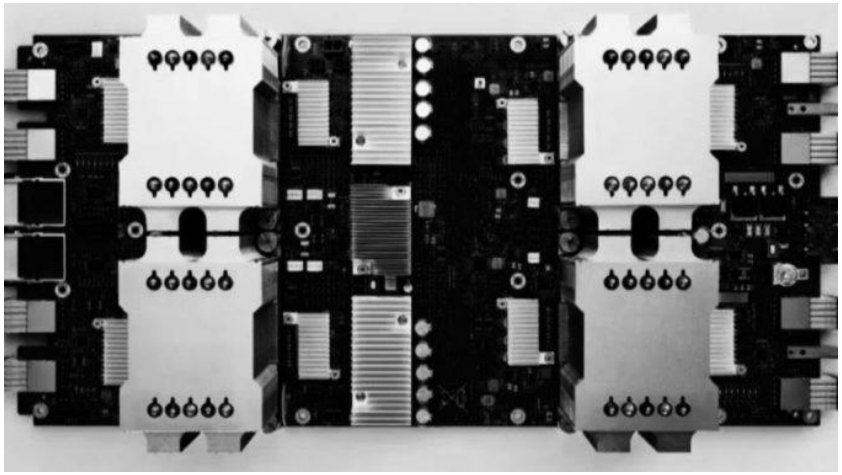
- Parallele Berechnungen möglich
- Kann durch Matrix-Multiplikationen formulieren werden
  → sehr gut geeignet für GPUs

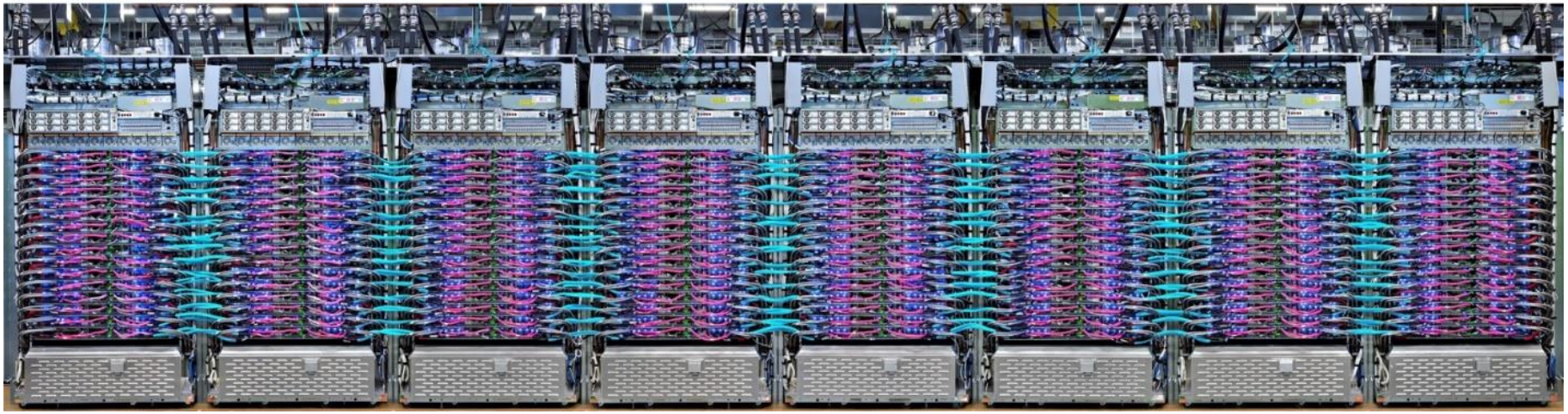
Nachteile:

- # der Schichten und Neurone pro Schicht ist Design-Entscheidung
  - Theoretisches Minimum von 2/3 Schichten
    (mehr ist unter umständen besser, z.B. schneller Konvergenz)
  - Bestimmung der Parameter heuristisch → durch Experimente
- Hoher Rechenaufwand nötig
- Optimales Minimum durch nicht-lineare Optimierung nicht garantiert

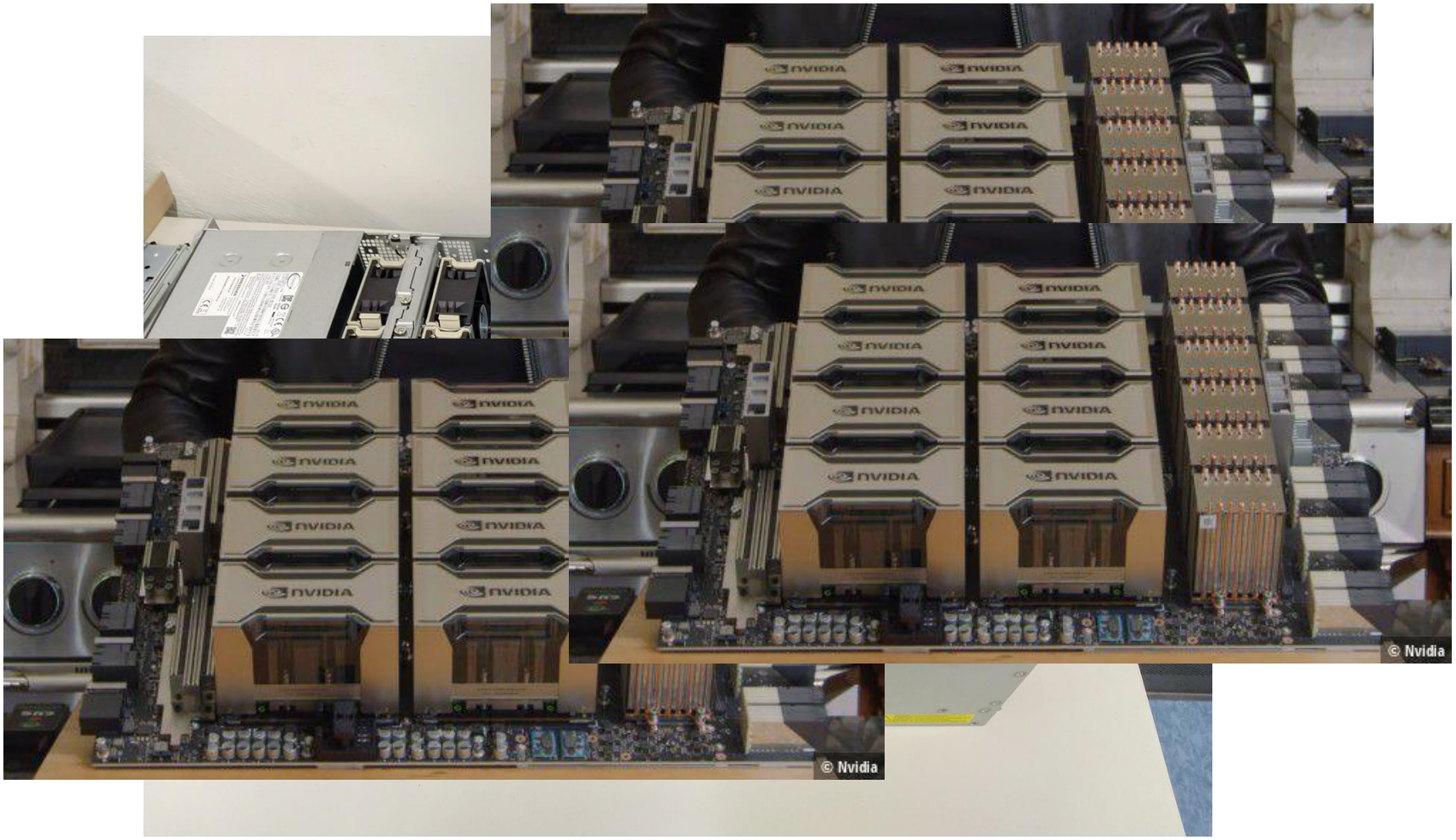
Paralleles Rechnen







Paralleles Rechnen



Backpropagation – Pseudocode

Für *N* Iterationen:

Feedforward: Laufe vorwärts durch das Netzwerk

Speichere Ausgaben der Neuronen vor und nach der Aktivierungsfunktion

Berechne Fehlerterm der letzten Schicht

Backpropagate Fehlerterm in vordere Schichten

Update Gewichte $\Delta w_i = -\varepsilon \frac{\partial C}{\partial w_i}$

Aktivierungsfunktionen

- Sigmoid
  - Kennen wir schon von der logistischen Regression

```
def sigmoid(x):
    return (1 / (1 + np.exp(-x)))
```

  - $g(x) = \frac{1}{1+e^{-x}}$
  - Smooth
  - Wertebereich: $[0,1]$ → Gradienten explodieren nicht
  - Saturierung an den Enden
  - Vanishing Gradient
  - Verwendung in Ausgabeschicht

Binäre Kreuzentropie als Kostenfunktion

- $C = \frac{1}{2M} \sum_{i=1}^{M} Cost(h(x^{(i)}) - y^{(i)})$

- $Cost(h(x), y) = \begin{cases} -\log(h(x)), & wenn\ y = 1 \\ -\log(1 - h(x)), & wenn\ y = 0 \end{cases}$

- *y immer 0 oder 1*

Binäre Kreuzentropie als Kostenfunktion

- $Cost(h(x), y) = -y \log\big(h(x)\big) - (1 - y) \log\big(1 - h(x)\big)$

- $C = \frac{1}{m} \sum_{i=1}^{m} Cost\big(h\big(x^{(i)}\big), y^{(i)}\big)$

$$= -\frac{1}{m} \sum_{i=1}^{m} y^{i} \log\big(h\big(x^{(i)}\big)\big) + \big(1 - y^{(i)}\big) \log\big(1 - h\big(x^{(i)}\big)\big)$$

Mehrklassen-Klassifikation

- Binäre Klassifikation: $y \in \{0, 1\} \quad oder \; y \in \{-1, 1\}$
- Mehrklassen-Klassifikation: $y \in \{0, 1, \dots, K\} \quad (\mathrm{K} = \text{Anzahl der Klassen})$
- Stattdessen: $\boldsymbol{y} = (y_1, \dots, y_K)^T$
- Ermöglicht Mehrklassen-Klassifikation

- Mit einer exklusiven Definition:

$$y_k = \begin{cases} 1, & wenn\ k\ Index\ der\ richtigen\ Klasse \\ 0, & sonst. \end{cases}$$

- Nennt man: **One-hot encoding**
- Ausgaben $\boldsymbol{y}$ können Wahrscheinlichkeiten modellieren (siehe gleich)

Categorial Crossentropy

- Erweiterung zum Mehrklassenproblem

$$C = -\sum_{k=1}^{K} y_k \log(h_k(x))$$

Softmax Aktivierungsfunktion

$$y_k = \frac{e^{x_k}}{\sum_{j=1}^{K} e^{x_j}}$$

- Auch *Normalisierte Exponentialfunktion* genannt
- Es gilt
  - $\sum_{k=1}^{K} y_k = 1$
  - $y_k \geq 0$
- Wahrscheinlichkeitsverteilung
- Ermöglicht Ausgaben als diskrete Wahrscheinlichkeitsfunktion zu betrachten

Softmax Aktivierungsfunktion

$$y_k = \frac{e^{x_k}}{\sum_{j=1}^{K} e^{x_j}}$$

- Beispiel:



Aktivierungsfunktion/Kostenfunktion – Klassifikation

Zwei Klassen

- Einzelnes Neuon in Ausgabeklasse
- Aktivierungsfunktion in der Ausgabeschicht: sigmoid
- Aktivierungsfunktion in verdeckten Schichten: ReLU
- Kostenfunktion: binary cross entropy

Mehrere Klassen

- Ein Neuron pro Klasse in Ausgabeschicht(1-out-of-n coding, one-hot) in output layer
- activation output layer: Softmax
- activation hidden layers: ReLU
- loss function: cross entropy

Activation/Loss Function – Regression

- Anstatt diskreter Klassen: Vorhersage eines kontinuierlichen Wertes

- Einzelnes Neuron in Ausgabeschicht
- Aktivierung in der Ausgabeschicht: Linear
  (Summe der Gewichte wird weitergereicht)
- Aktivierung in den verdeckten Schichten: ReLU
- Kostenfunktion: Mean Squared Error

Unterschiedliche Gradientenabstiegsverfahren

- Batch gradient descent
- Stochastischer Gradientenabstieg → Stochastic Gradient Descent
- Minibatch gradient descent

Minibatch GD:

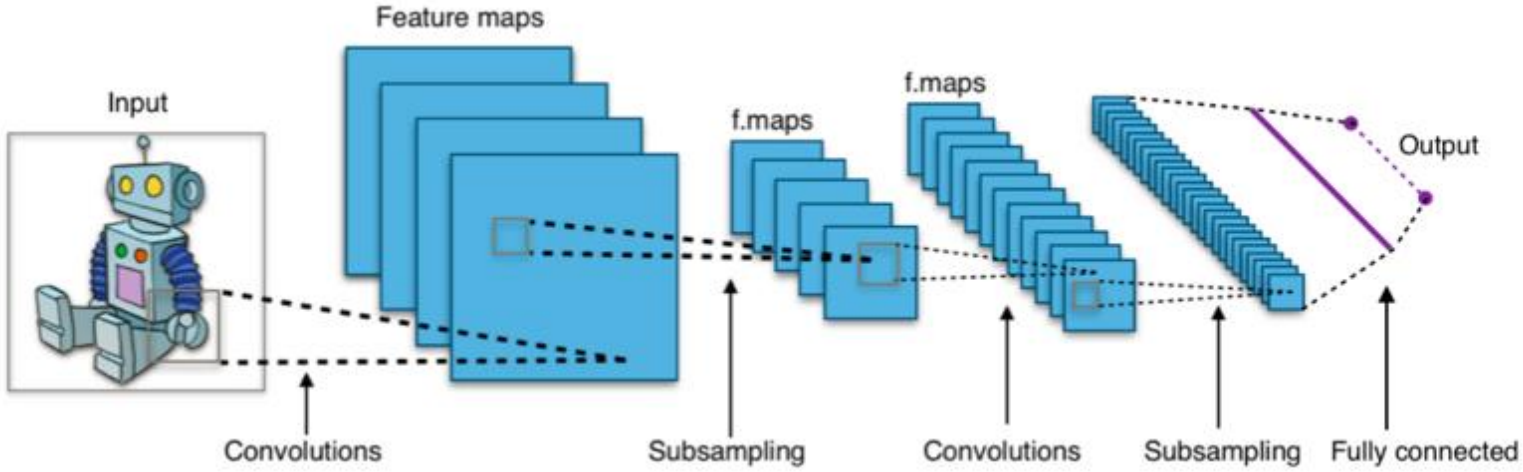
For each iteration k:

- select a small set of training samples randomly (batch)
- compute gradients
- update weights

Andere Varianten Neuronaler Netze

Convolutional Neural Networks (CNN)

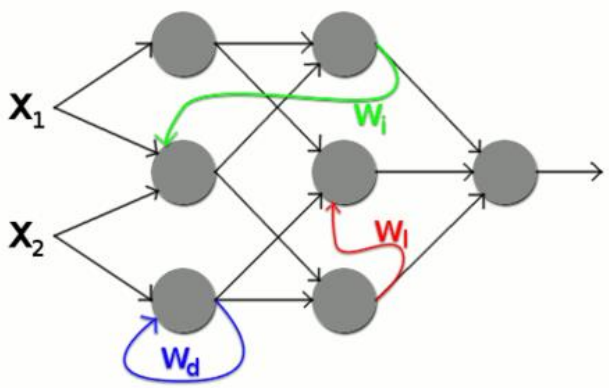
- Bildverarbeitung
- deep feed-forward networks
- Nicht fully-connected
- Integriert convolution-Operation (filter) in Netz

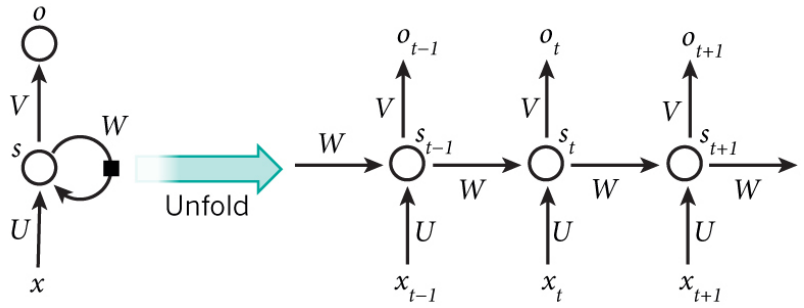


Andere Varianten Neuronaler Netze

Recurrent Neural Networks (RNN)

- Rekurrent, nicht feed-forward
- Speichert Zustand
- Verbindungen innerhalb einer Schicht a layer or to previous layers
- Sequence learning (Spracherkennung)
- Oft verwendet: Long Short-Term Memory (LSTM) networks





Zusammenfassung

- Neuronale Netze
- Motivation
- Geschichte
- Vom Perzeptron zum Neuronalen Netz
- Deep Nets
- Trainieren Neuronaler Netze: Backpropagation
- Mehrklassen-Probleme
- Softmax
- Weitere Netztopologien